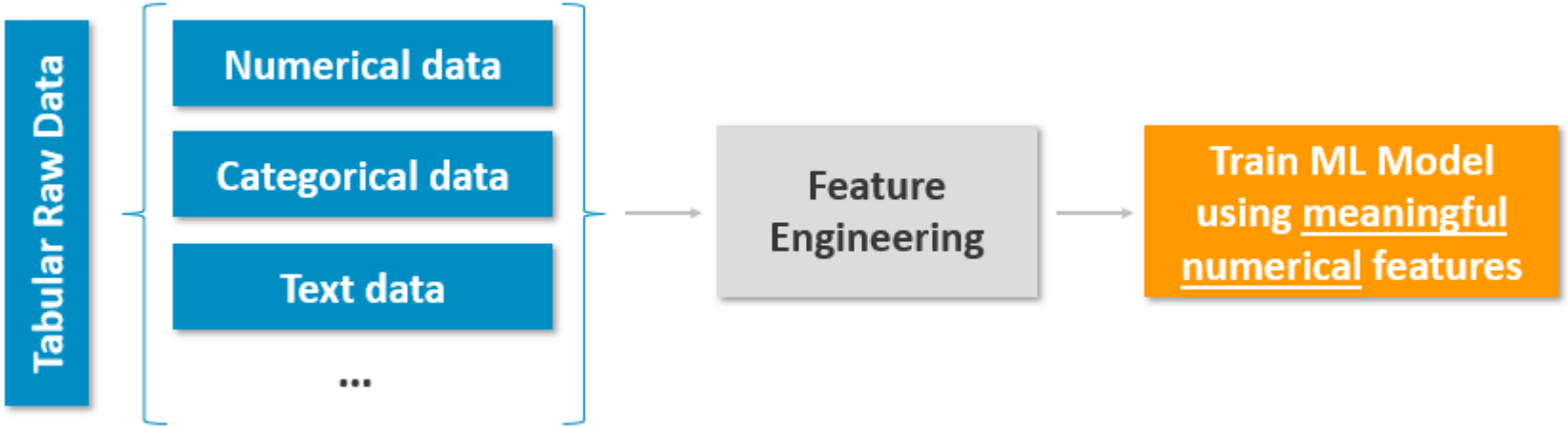
Ingeniería de características

Ingeniería de características

Utiliza conocimiento del **dominio y de los datos** para crear **nuevas características a partir de los datos crudos** como entradas para los modelos de ML.

Intuición: ¿Qué información utilizaría un humano para hacer predicciones?

¡A menudo, este proceso es más un arte que una ciencia!



Codificación de características

UNIVERSIDAD EAFIT | Escuela de Ciencias Aplicadas e Ingeniería

Características categóricas: No tienen una representación numérica natural.
Ejemplo: color Є {green, red, blue}, fraude Є {falso, verdadero}
La mayoría de modelos de ML requieren convertir estas categorías a números.

Codificación (encoding): Asignar un número a cada categoría.
Ordinal: Las categorías están ordenadas, ejemplo, tamaño Є {L>M>S}. Podemos asignar los valores L->3, M->2, S->1.

Nominal: Las categorías no tienen un orden específico, ejemplo, color Є {green, red, blue}. Podemos asignar números de forma aleatoria.

Codificación de características

UNIVERSIDAD EAFIT | Escuela de Ciencias Aplicadas e Ingeniería

LabelEncoder: **Clase de sklearn**, codifica variables categóricas con valores entre 0 y n_classes – 1. Implementa los métodos: .fit() y .transform()

Codifica las variables objetivo o una sola característica, no la matriz completa X.

	color	size	price	classlabel
0	green	S	10.1	shirt
1	red	M	13.5	pants
2	blue	L	15.3	shirt

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['color'] = le.fit_transform(df['color'])
```

	color	size	price	classlabel
0	1	S	10.1	shirt
1	2	M	13.5	pants
2	0	L	15.3	shirt

Codificación de características

UNIVERSIDAD EAFIT | Escuela de Ciencias Aplicadas e Ingeniería

`OrdinalEncoder`: **Clase de sklearn**, codifica variables categóricas con valores entre 0 y n_classes – 1. Implementa los métodos: `.fit()` y `.transform()`.

- Codifica dos o más variables categóricas. No funciona para una sola característica.
- Devuelve una sola columna de enteros por característica.

	color	size	price	classlabel
0	green	S	10.1	shirt
1	red	M	13.5	pants
2	blue	L	15.3	shirt

```
from sklearn.preprocessing import OrdinalEncoder
oe = OrdinalEncoder()

df[['color','size','classlabel']] =
oe.fit_transform(df[['color','size','classlabel']])
```

	color	size	price	classlabel
0	1.0	2.0	10.1	1.0
1	2.0	1.0	13.5	0.0
2	0.0	0.0	15.3	1.0

Codificación de características

UNIVERSIDAD EAFIT | Escuela de Ciencias Aplicadas e Ingeniería

Codificación usando la variable objetivo: Codificar utilizando valores que puedan explicar la variable objetivo.

Ejemplo: Promediar la variable objetivo para cada categoría. Después, reemplazar los valores categóricos con los valores promediados.

x_1	x_2	y
a	c	1
a	d	1
b	c	0
a	d	0
a	d	0
a	d	1
b	d	0

x_1 -> cat a -> 3/5 = 0.6
x_1 -> cat b -> 0/2 = 0
x_2 -> cat c -> 1/2 = 0.5
x_2 -> cat d -> 2/5 = 0.4

x_1	x_2	y
0.6	0.5	1
0.6	0.4	1
0	0.5	0
0.6	0.4	0
0.6	0.4	0
0.6	0.4	1
0	0.4	0

Transformaciones en sklearn

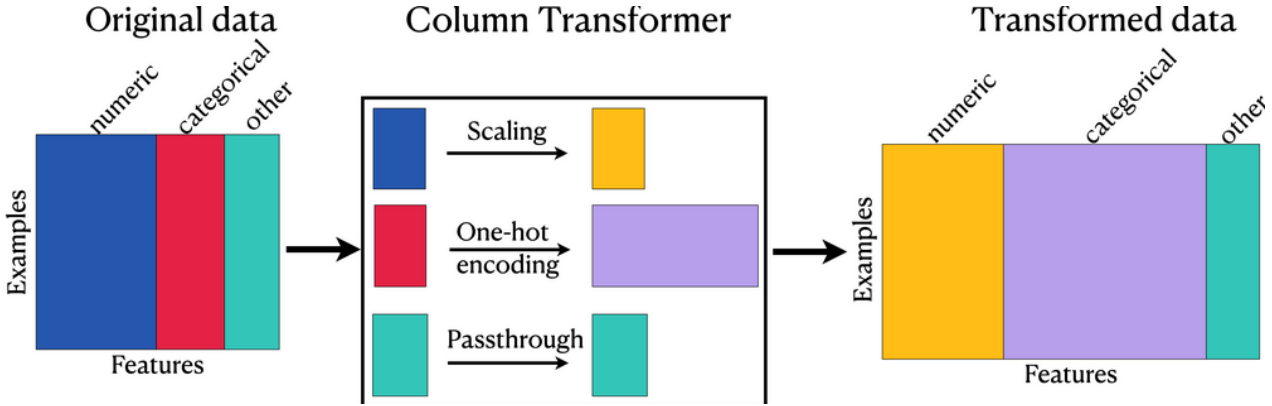
SimpleImputer, StandardScaler, MinMaxScaler, LabelEncoder, OrdinalEncoder, OneHotEncoder, CountVectorizer pertenecen a la clase Transformers de sklearn. Todos tienen los métodos:

- .fit(): aprende la transformación a partir de los datos de entrenamiento
- .transform() aplica la transformación a cualquier conjunto (entrenamiento, validación, prueba).
- En el conjunto de entrenamiento, también se puede aplicar .fit_transform()

ColumnTansformer en sklearn

`ColumnTransformer` aplica las transformaciones a las columnas de un arreglo de Numpy o a un DataFrame de Pandas. Implementa los métodos `.fit()` y `.transform()`.

- Permite aplicar transformaciones a subconjuntos de características (numérica, categóricas, texto), de forma individual.
- Las características generadas por cada transformación se concatenarán para conformar un solo grupo de características



ColumnTranformer y Pipeline

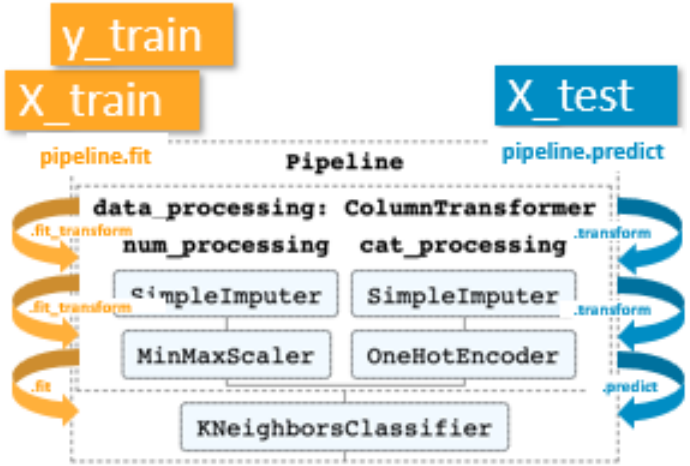
```
numerical_processing = Pipeline([
    ('num_imputer', SimpleImputer(strategy='mean')),
    ('num_scaler', MinMaxScaler())])

categorical_processing = Pipeline([
    ('cat_imputer', Imputer(strategy='constant', fill_value='missing')),
    ('cat_encoder', OneHotEncoder(handle_unknown='ignore'))])

processor = ColumnTransformer(transformers =[
     ('num_processing', numerical_processing, ('feature1', 'feature3')),
     ('cat_processing', categorical_processing, ('feature0', 'feature2'))])

pipeline = Pipeline([('data_processing', processor),
                     ('estimator', KNeighborsClassifier())])

pipeline.fit(X_train, y_train)
predictions = pipeline.predict(X_test)
```



Regresión Lineal, Logística

Notación matemática

UNIVERSIDAD EAFIT | Escuela de Ciencias Aplicadas e Ingeniería

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value	ocean_proximity
0	-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
2	-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
3	-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0	NEAR BAY
4	-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0	NEAR BAY

Vector de características

$$\mathbf{x}^{(i)} = \begin{bmatrix} x_1^{(i)} \\ x_2^{(i)} \\ \vdots \\ x_d^{(i)} \end{bmatrix} \in \Re^{d \times 1}$$

$$y^{(i)} \in \Re$$

$$y^{(i)} \in [0, 1, \cdots, C]$$

Matriz de características

$$\mathbf{X} = \begin{bmatrix} \cdots & \mathbf{x}^{(1)\top} & \cdots \\ \cdots & \mathbf{x}^{(2)\top} & \cdots \\ & \vdots & \\ \cdots & \mathbf{x}^{(n)\top} & \cdots \end{bmatrix} \in \Re^{n \times d} \qquad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

Métodos de aprendizaje de máquina

Formas de aproximar la relación entre $\mathbf{x}^{(i)}$ y $\mathbf{y}^{(i)}$

$$f(\mathbf{x}^{(i)}) = y^{(i)}$$

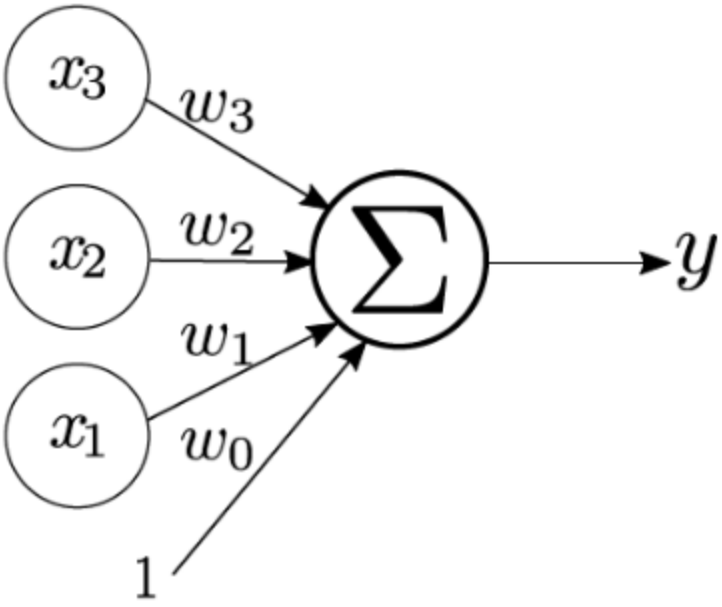
Función de costo

$$\mathcal{L}\left(f(\mathbf{x}^{(i)}), y^{(i)}\right)$$

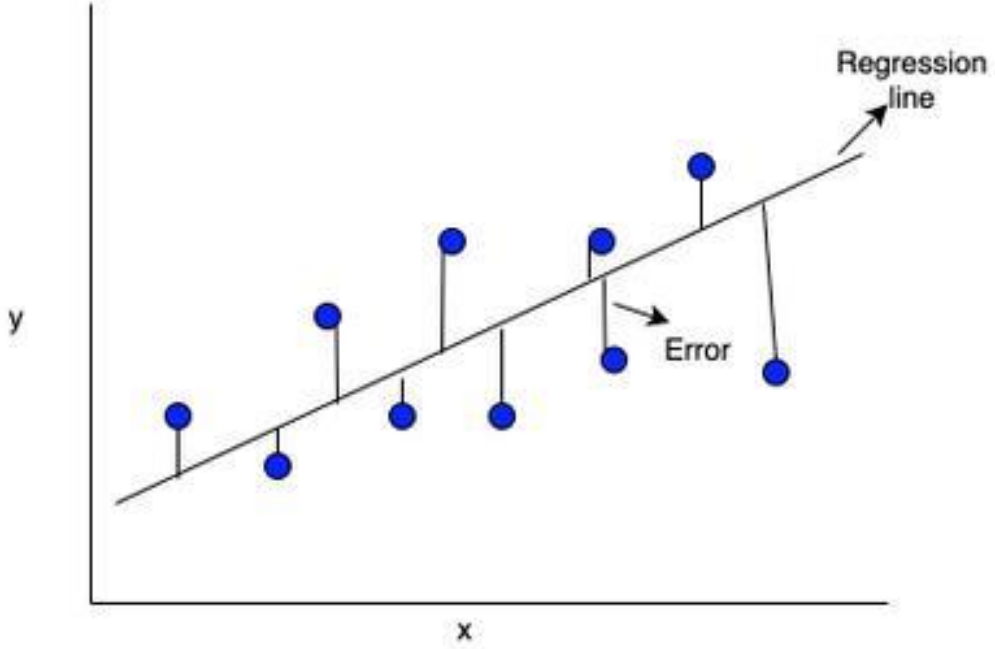
Regresión Lineal: Modelo

$$\hat{y} = \hat{f}(\mathbf{x}) = w_0 + \sum_{j=1}^{d} w_j \times x_j$$

$$\hat{y} = \mathbf{w}^T \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}$$



Regresión Lineal: Error en Función Objetivo



Regresión Lineal: Función Objetivo

$$Cost_{\mathbf{w}} = \frac{1}{m}\sum_{i=1}^{n}\left(y_i - \mathbf{w}^T\begin{bmatrix}\mathbf{x}_i \\ 1\end{bmatrix}\right)^2$$

Regresión Lineal: Entrenamiento

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

Descenso por el gradiente:

$$\mathbf{w} := \mathbf{w} - \alpha \times \nabla Cost_{\mathbf{w}}$$

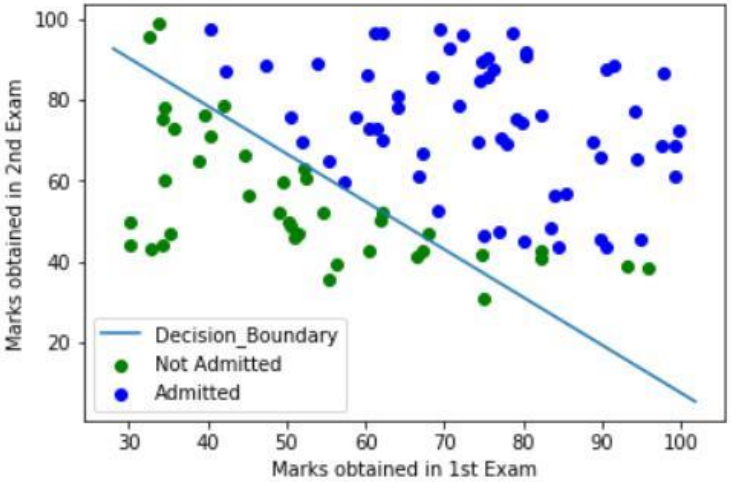
$$\nabla Cost_{\mathbf{w}} = \frac{1}{m} X^T (\hat{y} - y)$$

Ecuación normal:

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} (\mathbf{X}^T Y)$$

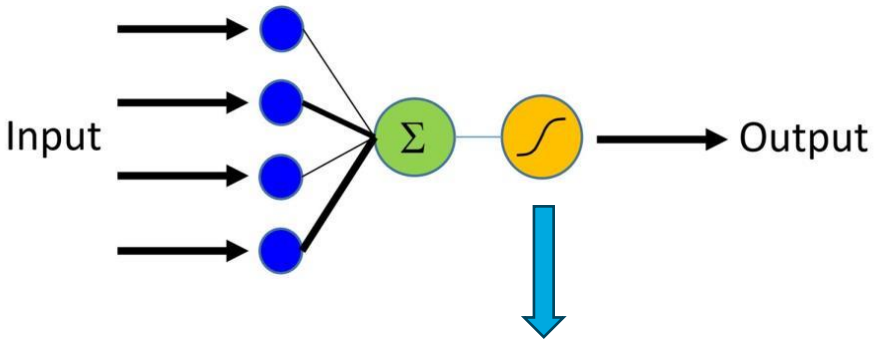
Regresión Logística: Nombre

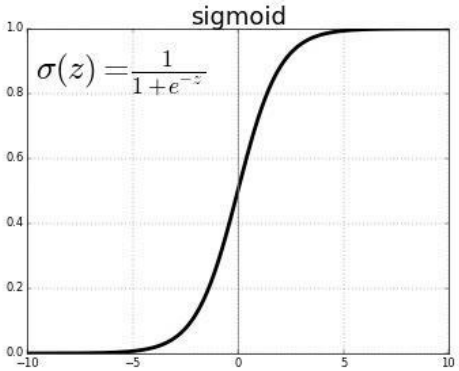
La regresión logística es un tipo de modelo lineal de **clasificación** a pesar de llevar históricamente la palabra regresión en su nombre.



Regresión Logística: Modelo

$$\hat{y} = h(\mathbf{x}) = \frac{1}{1 + \exp\left(-\mathbf{w}^T \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}\right)}$$





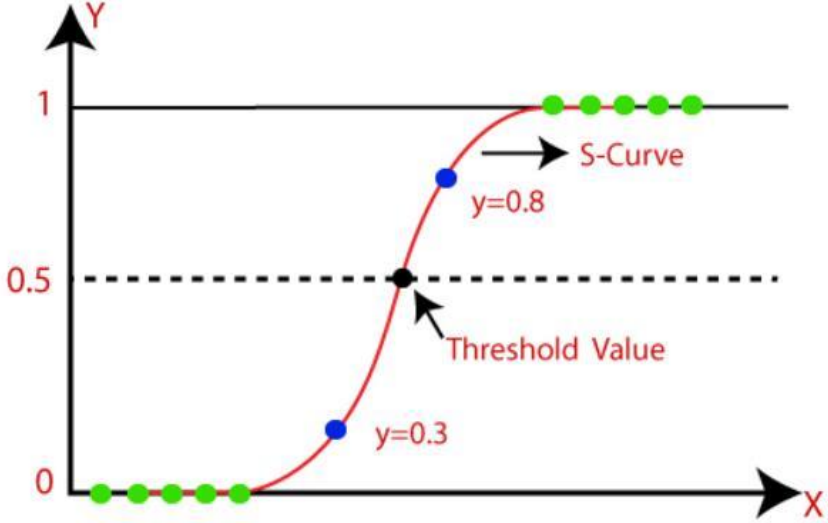
Regresión Logística: Etiqueta

La **etiqueta** con la que se entrena una regresión logística debe ser **binaria**, tomando únicamente los valores 0 o 1.

La clase 0 se suele denominar **clase negativa**, mientras que la clase 1 se suele llamar **clase positiva**.

Regresión Logística: Predicción

La salida que retorna una regresión logística es un número entre 0 y 1 que se interpreta como la probabilidad de que un ejemplo pertenezca a la clase positiva. La predicción final se decide según un umbral (por lo general 0.5)



Regresión Logística: Función Objetivo

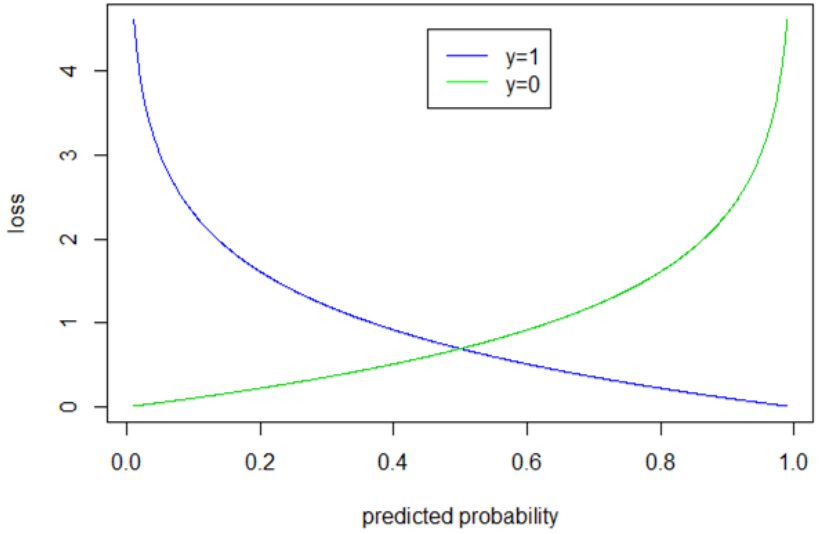
UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}[-y^{(i)}log(h_\theta(x^{(i)})) - (1-y^{(i)})log(1-h_\theta(x^{(i)}))]$$

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x_j^{(i)}$$

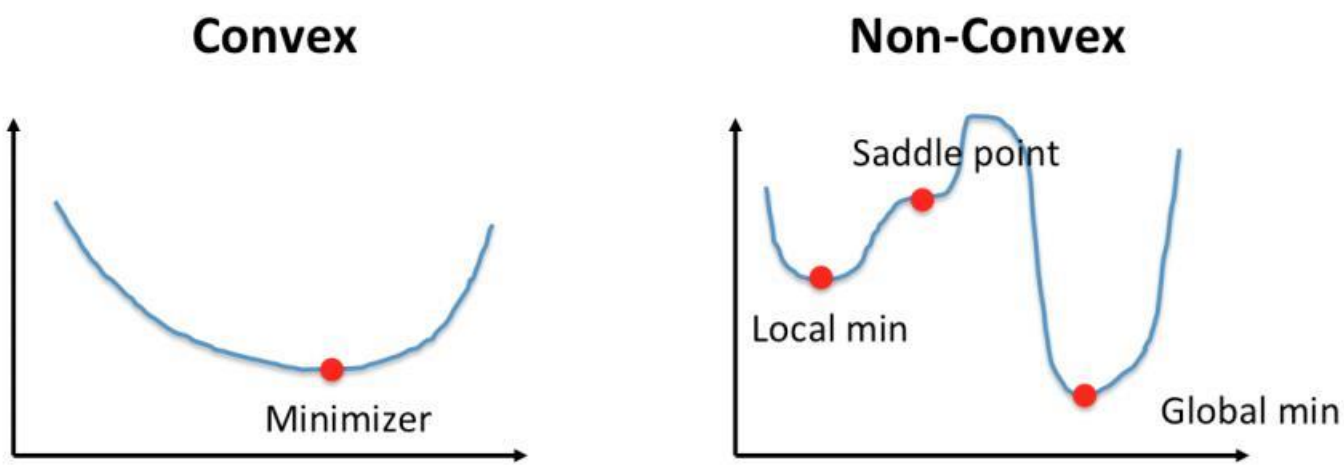
Regresión Logística: Función Objetivo

Esta función objetivo se denomina **pérdida de entropía cruzada (cross-entropy loss).**



Regresión Logística: Función Objetivo

La pérdida de entropía cruzada se utiliza porque permite que el problema de optimización para entrenar la regresión logística se vuelva **convexo.**



Regresión Logística: Entrenamiento

La regresión logística sólo se puede entrenar mediante métodos iterativos. Por lo general, se utiliza el descenso por el gradiente.

$$\mathbf{w} := \mathbf{w} - \alpha \times \nabla Cost_{\mathbf{w}}$$

$$\nabla Cost_{\mathbf{w}} = \frac{1}{m} X^T (\hat{y} - y)$$

GRACIAS